



**INSTITUTO POLITÉCNICO NACIONAL.
UNIDAD PROFESIONAL INTERDISCIPLINARIA EN
INGENIERÍA Y TECNOLOGÍAS AVANZADAS**



**Unidad de aprendizaje:
Bases de datos distribuidas.**

Tarea 3: Diseño de propuesta de fragmentación.

Alumnos:

- Fuentes García Liliana Alejandra.
- Marcelino Cerón Diego Antonio

Profesora:

Carlos De La Cruz Sosa.

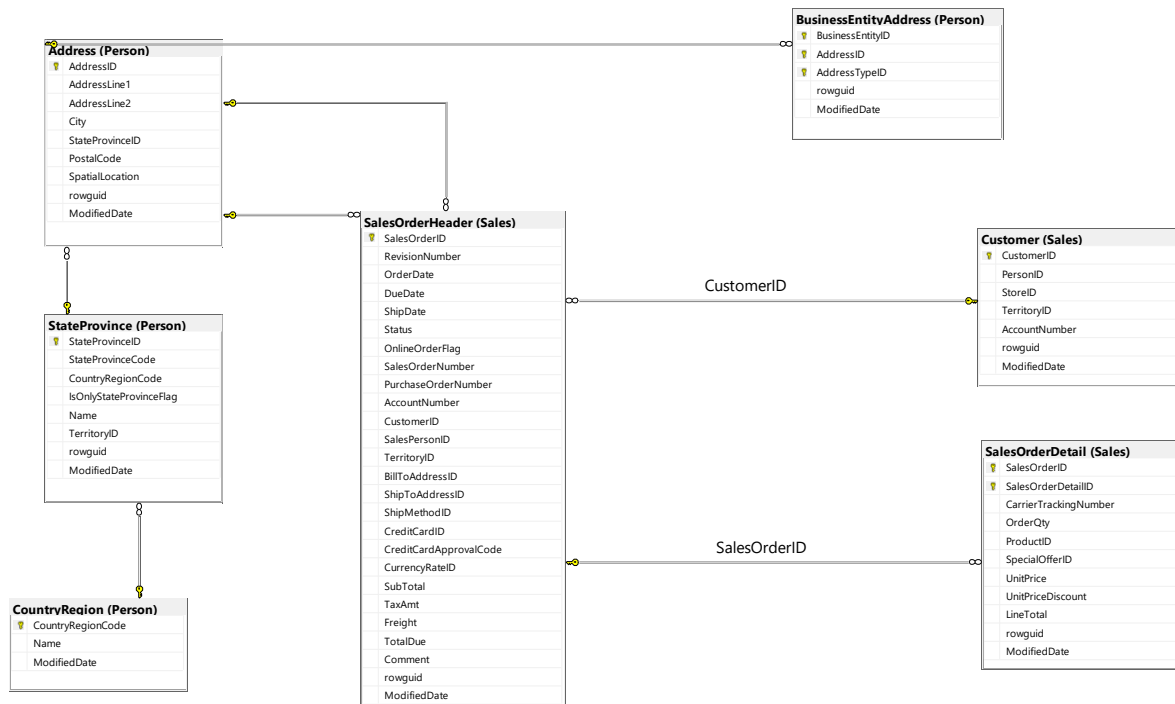
Fecha de entrega:

25 de septiembre del 2026.

Ejercicio seleccionado:

Ejercicio 3: Distribuir la cartera de clientes en los 5 nodos geográficos para que cada región administre y consulte de forma autónoma los datos de sus propios compradores. Tabla Sales.Customer, columna CustomerId(rangos).

Grafo relacional relacionado con el ejercicio seleccionado.



Se seleccionaron estas tablas porque permiten representar las relaciones principales de Sales.Customer dentro del alcance del ejercicio. Sales.SalesOrderHeader contiene las órdenes asociadas a cada cliente, y Sales.SalesOrderDetail contiene los productos de cada orden. La tabla Person.BusinessEntityAddress relaciona personas y tiendas con sus direcciones mediante BusinessEntityID. Person.Address, Person.StateProvince y Person.CountryRegion permiten seguir esa relación hasta la ubicación geográfica.

El diagrama muestra las tablas originales y las que pueden derivarse de los fragmentos de clientes mediante *semi-join*. La fragmentación primaria se aplica a Sales.Customer usando rangos de CustomerID. Los encabezados de órdenes y sus detalles se derivan con CustomerID y SalesOrderID, respectivamente. Para asociar direcciones se utilizan los valores PersonID y StoreID de cada fragmento, que se vinculan con Person.BusinessEntityAddress a través de BusinessEntityID. Las tablas de ubicación aportan contexto geográfico, aunque los rangos de CustomerID por sí solos no identifican la región de cada cliente.

COM_MIN y fragmentos primarios.

La relación propietaria seleccionada es Sales.Customer y el atributo de fragmentación es CustomerID.

Se necesitan cuatro predicados simples para definir cinco fragmentos:

$$Pr = \{p_1, p_2, p_3, p_4\}$$

Se propone dividir los clientes en cinco rangos, uno para cada nodo. Se definen cuatro límites ordenados, $b_1 < b_2 < b_3 < b_4$, y el conjunto de predicados simples.

$$\begin{aligned} p_1 : CustomerID \leq b_1, \quad p_2 : CustomerID \leq b_2, \\ p_3 : CustomerID \leq b_3, \quad p_4 : CustomerID \leq b_4 \end{aligned}$$

Donde los límites deben cumplir $b_1 < b_2 < b_3 < b_4$. Al combinar los predicados y sus negaciones, se obtienen los cinco fragmentos:

$$\begin{aligned} C_1 : CustomerID \leq b_1 \\ C_2 : b_1 < CustomerID \leq b_2 \\ C_3 : b_2 < CustomerID \leq b_3 \\ C_4 : b_3 < CustomerID \leq b_4 \\ C_5 : CustomerID > b_4 \end{aligned}$$

El quinto fragmento no necesita un quinto predicado simple: corresponde a los valores que no cumplen p_4 , es decir, $C_5 = \neg p_4$.

En este diseño, los rangos de CustomerID representan una distribución por identificador y no asignan por sí mismos clientes a regiones geográficas.

Los predicados no tienen por sí mismos valores mínimo y máximo: cada p_i establece un límite superior b_i . Los extremos pertenecen a los datos de Sales.Customer.

CustomerID va de 1 a 30118. El archivo de clientes de la muestra comienza con el identificador 1 y termina con 30118.

Para proponer cinco rangos numéricos de ancho parecido, el intervalo contiene $30118 - 1 + 1 = 30118$ valores enteros. Al dividirlo entre cinco, cada rango tendría aproximadamente 6024 valores. Esto da los límites:

	$b_1 = 6024,$	$b_2 = 12048,$	$b_3 = 18072,$	$b_4 = 24096$
Fragmento	Rango de <code>CustomerID</code>			
C_1	1–6024			
C_2	6025–12048			
C_3	12049–18072			
C_4	18073–24096			
C_5	24097–30118			

- **Aplicación de COM_MIN.**

El algoritmo COM_MIN analiza los predicados simples respecto a las consultas de las aplicaciones. Se conserva un predicado cuando permite distinguir el acceso de al menos una aplicación a los datos. Bajo el supuesto de que las cinco aplicaciones consultan por separado los cinco rangos definidos, el conjunto resultante es:

$$Pr' = \{p_1, p_2, p_3, p_4\}$$

CustomerID identifica a cada fila de Sales.Customer, por lo que permite separar la cartera de clientes en rangos sin dividir una misma fila entre varios fragmentos. Los cuatro predicados representan los límites entre esos rangos.

La selección supone que cada uno de los cinco nodos consulta principalmente el segmento de clientes que le corresponde. Bajo ese supuesto, cada predicado ayuda a distinguir qué datos necesita consultar cada aplicación. COM_MIN conserva los predicados que permiten diferenciar esos accesos; si algunas aplicaciones consultaran siempre juntas los mismos rangos, habría que reconsiderar si conviene mantenerlos separados.

Si las consultas no acceden de manera distinta a esos rangos, el conjunto debe ajustarse conforme al criterio de minimalidad.

- **Aplicación de fragmentación horizontal primaria.**

En esta parte se forman los predicados minterm combinando los predicados de Pr' con sus negaciones, elimina las combinaciones contradictorias y define los fragmentos primarios. Se combina los predicados de Pr' con sus negaciones para formar minterms. Como los predicados son límites ordenados, están anidados: si un cliente cumple p_1 , también cumple p_2 , p_3 y p_4 . Por eso, muchas combinaciones posibles son contradictorias y solo quedan cinco minterms válidos:

$$\begin{aligned}
m_1 &= p_1 \wedge p_2 \wedge p_3 \wedge p_4 \equiv p_1 \\
m_2 &= \neg p_1 \wedge p_2 \wedge p_3 \wedge p_4 \equiv \neg p_1 \wedge p_2 \\
m_3 &= \neg p_1 \wedge \neg p_2 \wedge p_3 \wedge p_4 \equiv \neg p_2 \wedge p_3 \\
m_4 &= \neg p_1 \wedge \neg p_2 \wedge \neg p_3 \wedge p_4 \equiv \neg p_3 \wedge p_4 \\
m_5 &= \neg p_1 \wedge \neg p_2 \wedge \neg p_3 \wedge \neg p_4 \equiv \neg p_4
\end{aligned}$$

Las equivalencias se simplifican porque los límites están ordenados y sus predicados son acumulativos. Por ejemplo, m_3 se simplifica a $\neg p_2$ **and** p_3 , que significa que el identificador supera 12048 y no supera 18072.

Los fragmentos primarios quedan definidos así:

Minterm	Fragmento primario
$m_1 = p_1$	$C_1 = \sigma_{m_1}(\text{Sales.Customer})$
$m_2 = \neg p_1 \wedge p_2$	$C_2 = \sigma_{m_2}(\text{Sales.Customer})$
$m_3 = \neg p_2 \wedge p_3$	$C_3 = \sigma_{m_3}(\text{Sales.Customer})$
$m_4 = \neg p_3 \wedge p_4$	$C_4 = \sigma_{m_4}(\text{Sales.Customer})$
$m_5 = \neg p_4$	$C_5 = \sigma_{m_5}(\text{Sales.Customer})$

En forma de rangos, C_1 contiene los identificadores hasta 6024; C_2 , los que superan 6024 y llegan hasta 12048; y así sucesivamente. C_5 contiene los que superan el último límite, hasta el máximo de la relación. De esta manera, los fragmentos no se traslapan y su unión reconstruye Sales.Customer.

Los fragmentos derivados de órdenes siguen siendo H_i , obtenidos mediante CustomerID, y D_i , obtenidos desde H_i mediante SalesOrderID. Para las direcciones, si decides mostrarlas como fragmentos derivados, sustituye Sales.CustomerAddress por Person.BusinessEntityAddress. El vínculo debe pasar por los identificadores PersonID o StoreID de los clientes y relacionarlos con BusinessEntityID; no se une directamente mediante CustomerID.

Fragmentación horizontal derivada(semi-join).

Los fragmentos derivados se obtienen mediante *semi-join*, siguiendo las relaciones del subgrafo relacional:

$$H_i = \text{Sales.SalesOrderHeader} \bowtie C_i$$

La condición del *semi-join* es:

$$\text{Sales.SalesOrderHeader.CustomerID} = \text{Sales.Customer.CustomerID}$$

A partir de los encabezados de órdenes se obtienen los detalles:

$$D_i = \text{Sales.SalesOrderDetail} \bowtie H_i$$

La condición del segundo *semi-join* es:

$$\text{Sales.SalesOrderDetail.SalesOrderID} = \text{Sales.SalesOrderHeader.SalesOrderID}$$

También puede derivarse la asociación entre las entidades de los clientes de C_i y sus direcciones. Para ello, primero se consideran los valores no nulos de PersonID y StoreID de C_i , que identifican personas o tiendas. Esos identificadores se relacionan con Person.BusinessEntityAddress.BusinessEntityID:

$$BEA_i = \text{Person.BusinessEntityAddress} \bowtie \text{Entidades}(C_i)$$

donde Entidades(C_i) representa los PersonID y StoreID asociados con los clientes del fragmento C_i . La condición de unión es:

$$\text{Person.BusinessEntityAddress.BusinessEntityID} = \text{Entidades.BusinessEntityID}$$

Person.BusinessEntityAddress relaciona las entidades con Person.Address mediante AddressID; Person.Address se relaciona con Person.StateProvince mediante StateProvinceID, y esta última con Person.CountryRegion mediante CountryRegionCode. Estas tablas permiten mostrar la ruta hacia la ubicación, pero no se consideran fragmentos disjuntos en esta propuesta.

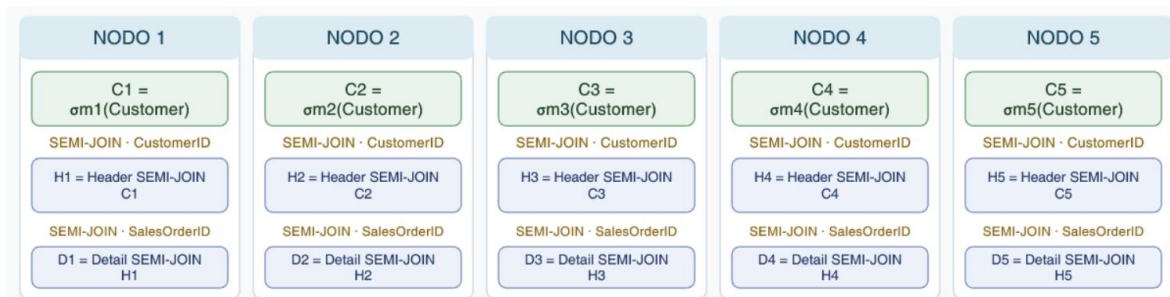
Por tanto, para cada $i=1, \dots, 5$, se obtienen H_i , D_i y BEA_i asociados con C_i . Los fragmentos de órdenes se derivan mediante CustomerID y SalesOrderID; la asociación de direcciones se deriva mediante BusinessEntityID.

Un *semi-join* conserva las filas de una tabla que tienen una coincidencia en otra, usando la clave relacionada. Aplicado a la propuesta:

- H_i contiene los encabezados de Sales.SalesOrderHeader correspondientes a los clientes de C_i . Se relacionan mediante CustomerID.
- D_i contiene los detalles de Sales.SalesOrderDetail correspondientes a las órdenes de H_i . Se relacionan mediante SalesOrderID.
- BEA_i contiene las asociaciones de Person.BusinessEntityAddress correspondientes a las personas o tiendas de los clientes de C_i . Se relacionan mediante BusinessEntityID, obtenido de los valores no nulos de PersonID y StoreID de C_i .

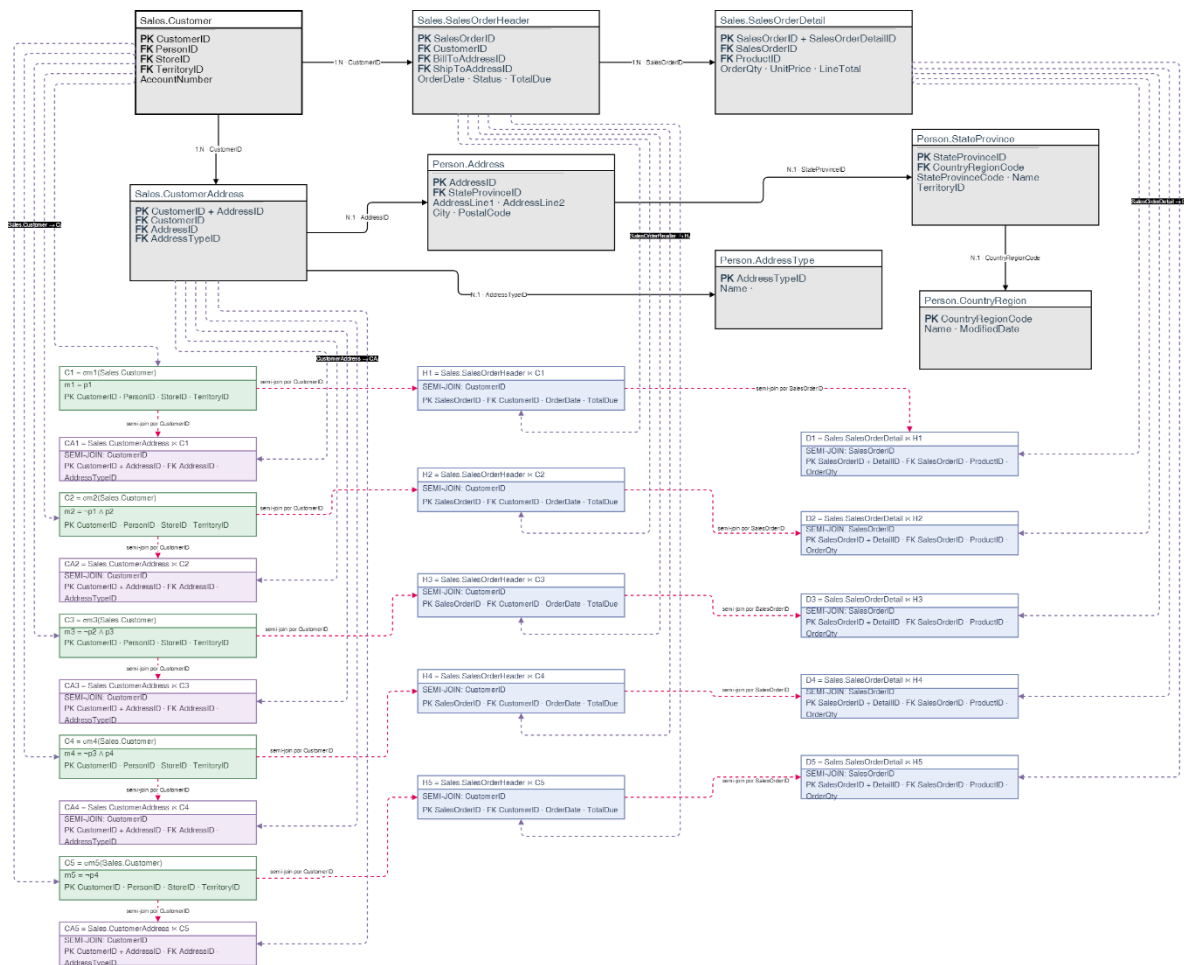
El proceso se realiza para cada nodo: primero se define el fragmento primario C_i ; con este se obtiene H_i ; a partir de H_i se obtiene D_i ; y, si se incluyen las asociaciones de dirección en la distribución, también se obtiene BEA_i desde C_i , pasando por los identificadores de persona o tienda. Así, cada fragmento derivado contiene las filas relacionadas con los clientes u órdenes del fragmento

Resumen de fragmentos por nodo:



El esquema resume la distribución lógica en cinco nodos. En cada uno, C_i representa el fragmento primario de clientes; H_i contiene los encabezados de órdenes relacionados mediante CustomerID, y D_i contiene los detalles asociados mediante SalesOrderID.

A continuación se presenta el subgrafo relacional seleccionado para la propuesta y su diseño de fragmentación. La figura muestra las tablas originales, sus relaciones y cómo los fragmentos primarios de Sales.Customer se propagan hacia las tablas relacionadas mediante *semi-join*.



En la parte superior se muestran las tablas originales y sus atributos principales; en la parte inferior se representa la fragmentación primaria de Sales.Customer y la derivación de las tablas relacionadas mediante semi-join. Las líneas identifican las relaciones del esquema y la propagación propuesta hacia los fragmentos.

Validación del diseño.

- **Completitud:** Los cinco rangos cubren todos los valores de CustomerID. Los fragmentos derivados incluyen las filas relacionadas con los fragmentos de clientes correspondientes.
- **Reconstrucción:** $\text{Sales.Customer} = C_1 \cup C_2 \cup C_3 \cup C_4 \cup C_5$. Las relaciones derivadas se reconstruyen uniendo todos sus fragmentos.
- **Disyunción:** Para $i \neq j$, $C_i \cap C_j = \emptyset$. La misma propiedad se comprueba para H_i , D_i y CA_i , según las claves de derivación.

Conclusión.

La propuesta fragmenta horizontalmente Sales.Customer en cinco partes mediante rangos de CustomerID. A partir de esos fragmentos se derivan los encabezados de órdenes de Sales.SalesOrderHeader mediante CustomerID y los detalles de Sales.SalesOrderDetail mediante SalesOrderID. Para representar las asociaciones de direcciones se utiliza Person.BusinessEntityAddress, vinculada a los clientes por sus identificadores PersonID o StoreID y la clave BusinessEntityID; Person.Address, Person.StateProvince y Person.CountryRegion se mantienen como tablas relacionadas de referencia.

El diseño cumple completitud, reconstrucción y disyunción si los límites abarcan todos los valores de CustomerID y cada entidad asociada queda en el fragmento correspondiente. Como CustomerID distribuye por identificador, los nodos representan rangos de clientes y no regiones geográficas reales.